MODULE 4

Dimensionality reduction

- Dimensionality reduction is an important approach in machine learning.
- A large number of features available in the dataset may result in overfitting of the learning model.
- To identify the set of significant features and to reduce the dimension of the dataset, dimensionality reduction techniques that are used.

- Dimensionality reduction technique can be defined as, ***"It is a way of converting the higher dimensions dataset into lesser dimensions dataset ensuring that it provides similar information."***
- These techniques are widely used in machine learning for obtaining a better fit predictive model while solving the classification and regression problems.

- The number of input features, variables, or columns present in a given dataset is known as dimensionality.
- Process to reduce these features is called dimensionality reduction.
- A dataset contains a huge number of input features in various cases, which makes the predictive modeling task more complicated.
- Because it is very difficult to visualize or make predictions for the training dataset with a high number of features, for such cases, dimensionality reduction techniques are required to use.

Benefits of applying Dimensionality Reduction

- By reducing the dimensions of the features, the space required to store the dataset also gets reduced.
- Less Computation training time is required for reduced dimensions of features.
- Reduced dimensions of features of the dataset help in visualizing the data quickly.
- **I**t removes the redundant features (if present) by taking care of multi-collinearity.

Disadvantages of dimensionality Reduction

- Some data may be lost due to dimensionality reduction.
- In the PCA dimensionality reduction technique, sometimes the principal components required to consider are unknown.

Approaches of Dimension Reduction

- There are two ways to apply the dimension reduction technique:

Feature Selection

- Feature selection is the process of selecting the subset of the relevant features and leaving out the irrelevant features present in a dataset to build a model of high accuracy.
- In other words, it is a way of selecting the optimal features from the input dataset.
- Three methods are used for the feature selection:

1. Filters Methods

- In this method, the dataset is filtered, and a subset that contains only the relevant features is taken. Some common techniques of filters method are:
- **Correlation**
- **Chi-Square Test**
- **ANOVA**
- **Information Gain, etc.**

2. Wrappers Methods

- The wrapper method has the same goal as the filter method, but it takes a machine learning model for its evaluation.
- In this method, some features are fed to the ML model, and evaluate the performance.
- The performance decides whether to add those features or remove to increase the accuracy of the model.
- This method is more accurate than the filtering method but complex to work. Some common techniques of wrapper methods are:
- Forward Selection
- Backward Selection
- Bi-directional Elimination

3. Embedded Methods: Embedded methods check the different training iterations of the machine learning model and evaluate the importance of each feature.

Some common techniques of Embedded methods are:

- **LASSO**
- **Elastic Net**
- **Ridge Regression, etc.**

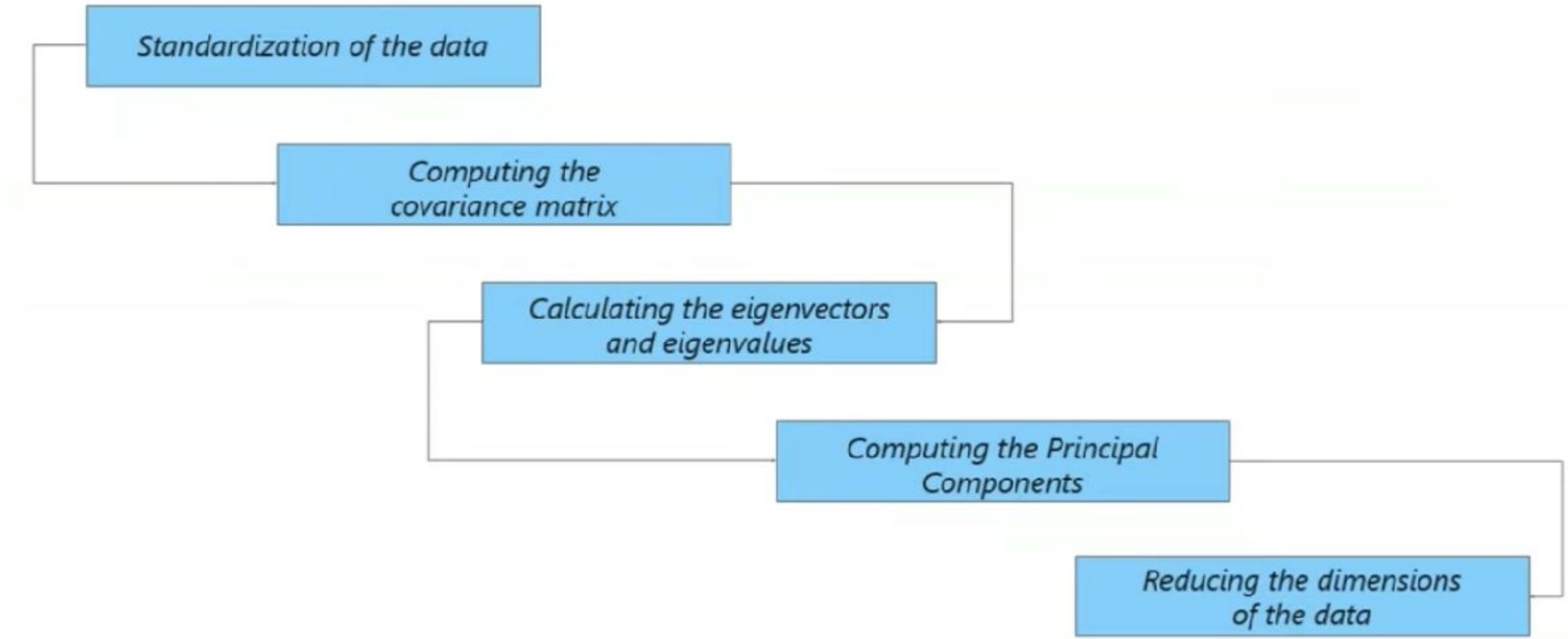
Feature Extraction:

- Feature extraction is the process of transforming the space containing many dimensions into space with fewer dimensions.
- This approach is useful when we want to keep the whole information but use fewer resources while processing the information.
- Some common feature extraction techniques are:
- Principal Component Analysis
- Linear Discriminant Analysis
- Kernel PCA
- Quadratic Discriminant Analysis

PCA – Principal Component Analysis

- Principal Component Analysis is a statistical process
- Converts the observations of correlated features into a set of linearly uncorrelated features with the help of orthogonal transformation.
- These new transformed features are called the **Principal Components**.
- It is one of the popular tools that is used for exploratory data analysis and predictive modeling.
- PCA works by considering the variance of each attribute because the high attribute shows the good split between the classes, and hence it reduces the dimensionality.
- Some real-world applications of PCA are ***image processing, movie recommendation system, optimizing the power allocation in various communication channels.***

STEP BY STEP PCA



1. Take the whole dataset consisting of d-dimensional samples ignoring the class labels
2. Compute the d-dimensional mean vector (i.e., the means for every dimension of the whole dataset)
3. Compute the scatter matrix (alternatively, the covariance matrix) of the whole data set
4. Compute eigenvectors ($\boldsymbol{e}_1, \boldsymbol{e}_2, \ldots, \boldsymbol{e}_d$) and corresponding eigenvalues ($\boldsymbol{\lambda}_1, \boldsymbol{\lambda}_2, \ldots, \boldsymbol{\lambda}_d$)
5. Sort the eigenvectors by decreasing eigenvalues and choose k eigenvectors with the largest eigenvalues to form a $d \times k$ dimensional matrix $\boldsymbol{W}$ (where every column represents an eigenvector)
6. Use this $d \times k$ eigenvector matrix to transform the samples onto the new subspace. This can be summarized by the mathematical equation: $\boldsymbol{y} = \boldsymbol{W}^T \times \boldsymbol{x}$ (where $\boldsymbol{x}$ is a $d \times 1$-dimensional vector representing one sample, and $\boldsymbol{y}$ is the transformed $k \times 1$-dimensional sample in the new subspace.)

LDA

- Linear Discriminant Analysis (LDA) is most commonly used as dimensionality reduction technique
- Used in the pre-processing step for pattern-classification and machine learning applications.
- The goal is to project a dataset onto a lower-dimensional space with good class-separability in order avoid overfitting and also reduce computational costs.

1. Compute the d-dimensional mean vectors for the different classes from the dataset.
2. Compute the scatter matrices (in-between-class and within-class scatter matrix).
3. Compute the eigenvectors ($\boldsymbol{e}_1, \boldsymbol{e}_2, \ldots, \boldsymbol{e}_d$) and corresponding eigenvalues ($\boldsymbol{\lambda}_1, \boldsymbol{\lambda}_2, \ldots, \boldsymbol{\lambda}_d$) for the scatter matrices.
4. Sort the eigenvectors by decreasing eigenvalues and choose k eigenvectors with the largest eigenvalues to form a $d \times k$ dimensional matrix $\boldsymbol{W}$ (where every column represents an eigenvector).
5. Use this $d \times k$ eigenvector matrix to transform the samples onto the new subspace. This can be summarized by the matrix multiplication: $\boldsymbol{Y} = \boldsymbol{X} \times \boldsymbol{W}$ (where $\boldsymbol{X}$ is a $n \times d$-dimensional matrix representing the n samples, and $\boldsymbol{y}$ are the transformed $n \times k$-dimensional samples in the new subspace).

Independent component Analysis

- A technique for revealing the hidden structure of data
- ICA model
  - The observed data variables are linear mixtures of latent variables
  - The mixing process is unknown
  - The latent variables are assumed to be non-Gaussian and statistically independent; they are called independent components
- ICA can find independent components

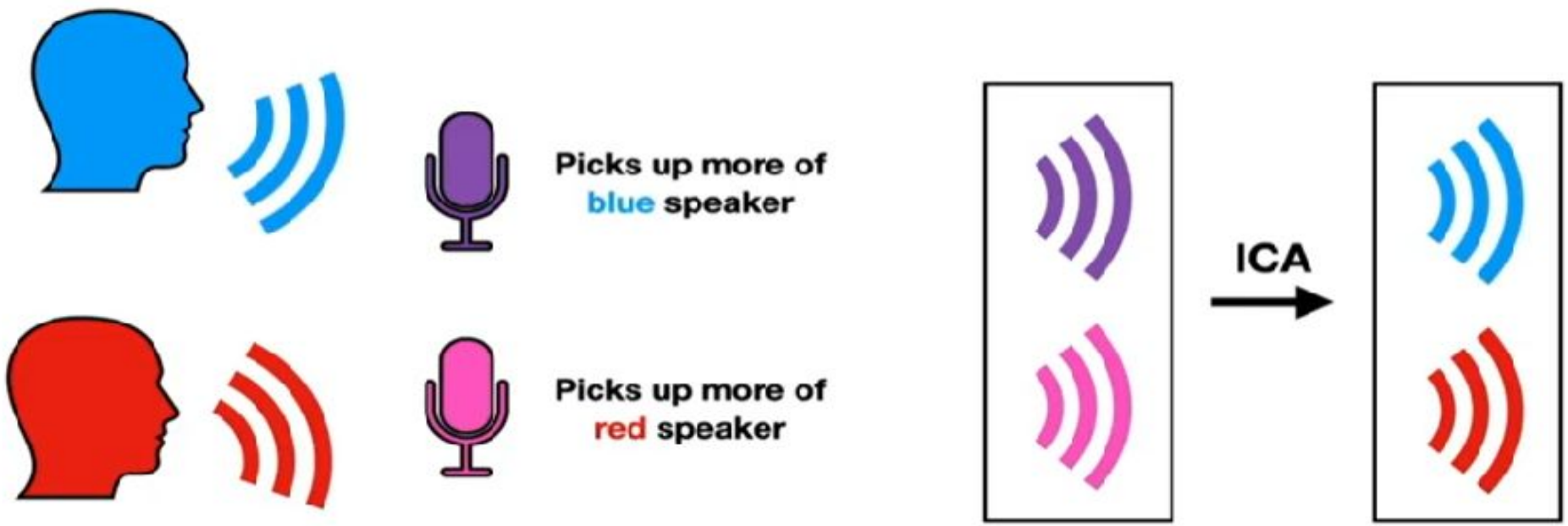
Independent Component Analysis (ICA)

Transforms set of vectors into maximally independent set



Independent Component Analysis (ICA)

Transforms set of vectors into maximally independent set



Independent Component Analysis (ICA)

Transforms set of vectors into maximally independent set

Assumptions: Independent Components are,

1. Statistically independent $p(x, y) = p(x)p(y)$
2. Non-gaussian

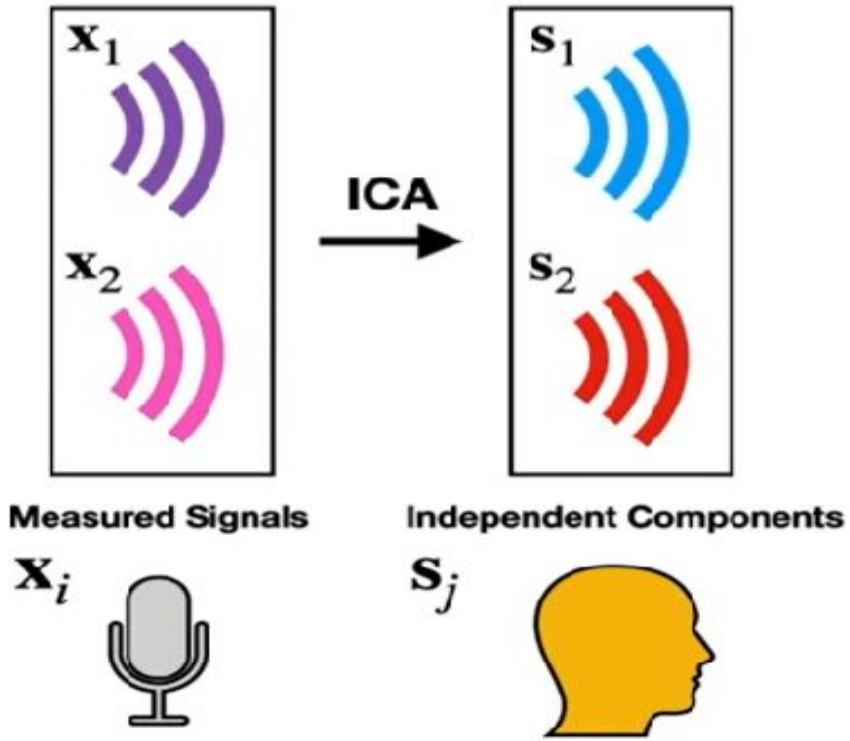


$$\mathbf{x}_i = \sum_j a_{ij}\mathbf{s}_j$$

or $$\mathbf{s}_i = \sum_j w_{ij}\mathbf{x}_j$$

Goal: Given $\mathbf{X}$ solve for $\mathbf{W}$ such that $\{\mathbf{s}_i\}$ are maximally independent

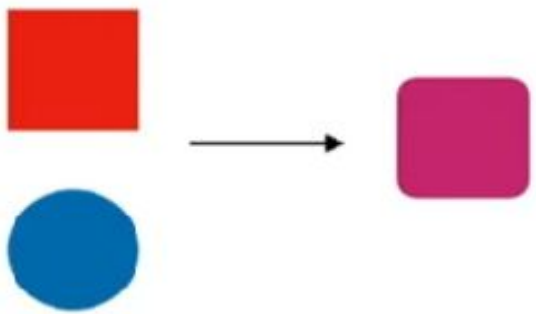
e.g. min mutual information or max non-Gaussianity (kurtosis, negentropy)



PCA & ICA

PCA

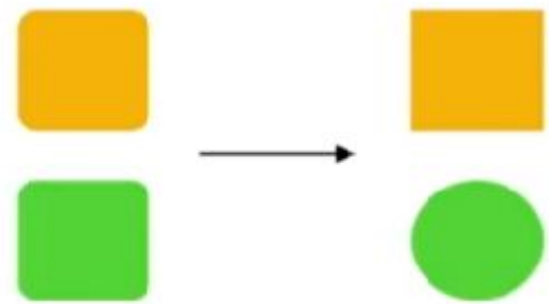
Compresses information



Requires preprocessing: autoscaling

ICA

Separates information

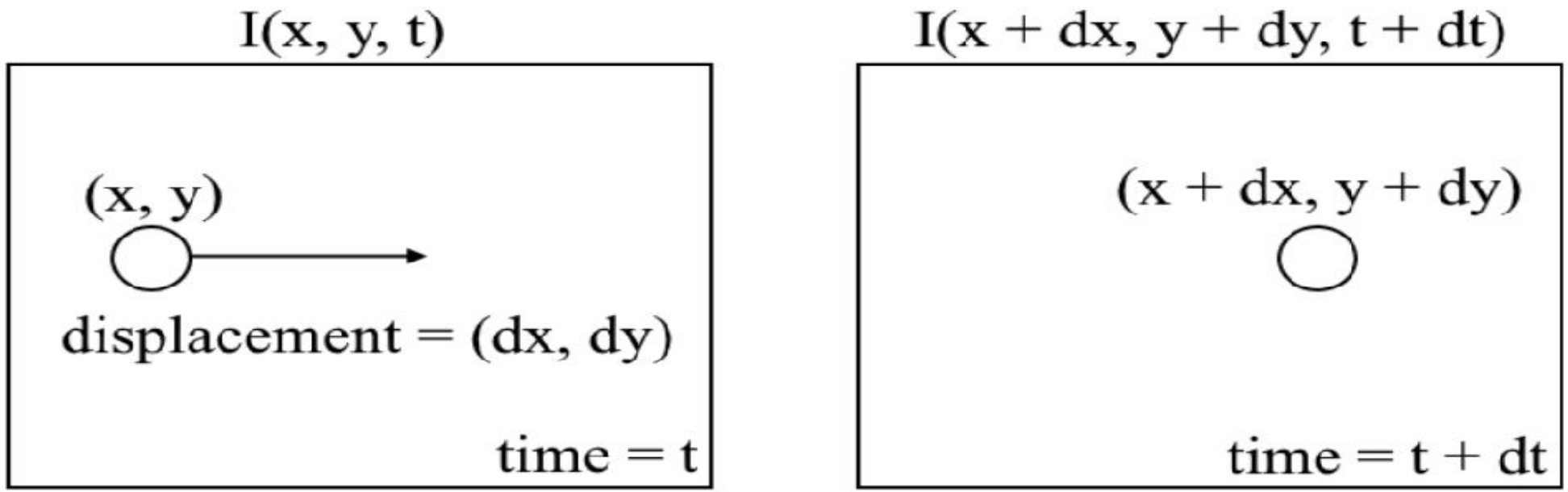


Requires preprocessing: autoscaling

Often benefits from first applying PCA

What is optical flow?

Let us begin with a high-level understanding of optical flow. Optical flow is the motion of objects between consecutive frames of sequence, caused by the relative movement between the object and camera. The problem of optical flow may be expressed as:



- between consecutive frames,
- we can express the image intensity (I) as a function of space (x,y) and time (t).
- In other words, if we take the first image I(x,y,t) and move its pixels by (dx,dy) over t time,
- we obtain the new image I(x+dx,y+dy,t+dt).

First, we assume that pixel intensities of an object are constant between consecutive frames.

$$I(x, y, t) = I(x + \delta x, y + \delta y, t + \delta t)$$

Constant intensity assumption for optical flow

Second, we take the Taylor Series Approximation of the RHS and remove common terms.

$$I(x + \delta x, y + \delta y, t + \delta t) = I(x, y, t) + \frac{\partial I}{\partial x}\delta x + \frac{\partial I}{\partial y}\delta y + \frac{\partial I}{\partial t}\delta t + \dots$$

$$\Rightarrow \frac{\partial I}{\partial x}\delta x + \frac{\partial I}{\partial y}\delta y + \frac{\partial I}{\partial t}\delta t = 0$$

Taylor Series Approximation of pixel intensity

Third, we divide by dt to derive the optical flow equation:

$$\frac{\partial I}{\partial x}u + \frac{\partial I}{\partial y}v + \frac{\partial I}{\partial t} = 0$$

Optical flow equation

where $u = dx/dt$ and $v = dy/dt$.

$dI/dx, dI/dy$, and dI/dt are the image gradients along the horizontal axis, the vertical axis, and time. Hence, we conclude with the problem of optical flow, that is, solving $u(dx/dt)$ and $v(dy/dt)$ to determine movement over time. You may notice that we cannot directly solve the optical flow equation for u and v since there is only one equation for two unknown variables. We will implement some methods such as the Lucas-Kanade method to address this issue.

Introduction

- Motion Tracking is recognizing a salient feature and monitoring its motion across multiple frames.
- Applications like facial detection, video surveillance, traffic estimation, etc.
- Object is monitored for spatial and temporal changes through a sequence of frames

Challenges

- Rapid movement of object across frames
- Changing object orientation
- Changing Illumination
- Complex feature tracking like facial expressions
- Interfering background

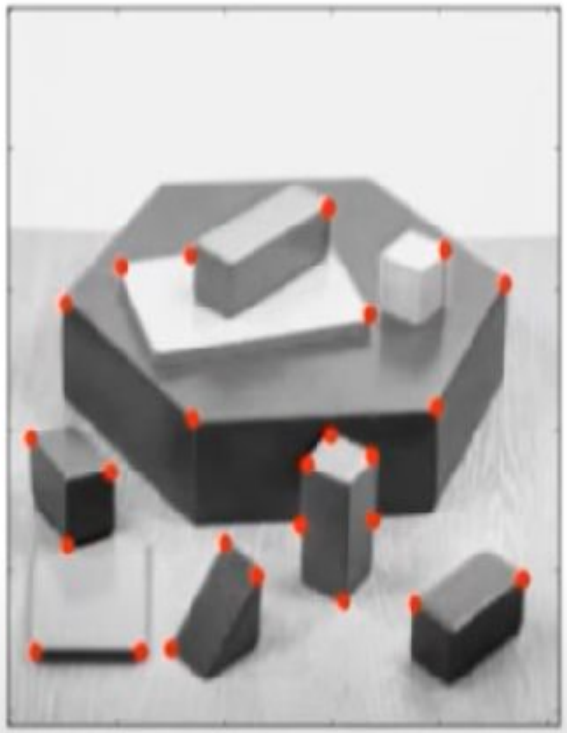
Methodology for Object tracking

Step 1 : Use a suitable feature detection algorithm to detect salient features in an image, that will then be tracked. In this project, Harris Corner Detector

Step 2: Optical Flow computation using Lucas-Kanade algorithm

Harris Corner Detector

- Corner is an intersection of two edges
- Harris Corner gives a mathematically representation for this concept.



$$\hat{M}(x,y) = \sum_{x,y} w(x,y) \otimes \begin{bmatrix} \left(\frac{\partial I}{\partial x}\right)^2 & \frac{\partial I}{\partial x}\cdot\frac{\partial I}{\partial y} \\ \frac{\partial I}{\partial x}\cdot\frac{\partial I}{\partial y} & \left(\frac{\partial I}{\partial y}\right)^2 \end{bmatrix}$$

Harris Corner Detector

- Cornerness value greater than a threshold is a Corner.

$$R = \det M - k(trace\, M)^2$$

$$\det M = \lambda_1 \lambda_2$$

$$trace\, M = \lambda_1 + \lambda_2$$

Algorithm

Step 1 : Find the image x and y derivatives, Ix and Iy.

$$I_x = G_\sigma^x * I \quad I_y = G_\sigma^y * I$$

Step 2 : Using Ix and Iy, find Ix2, Iy2 and Ixy.

$$I_{x2} = I_x . I_x \quad I_{y2} = I_y . I_y \quad I_{xy} = I_x . I_y$$

Step 3 : For every pixel, find the sum of product of derivatives.

$$S_{x2} = G_{\sigma\prime} * I_{x2} \quad S_{y2} = G_{\sigma\prime} * I_{y2} \quad S_{xy} = G_{\sigma\prime} * I_{xy}$$

Step 4 : Find the Cornerness using the Harris Corner Detector Equation.

$$\hat{M}(x,y) = \sum_{x,y} w(x,y) \otimes \begin{bmatrix} \left(\frac{\partial I}{\partial x}\right)^2 & \frac{\partial I}{\partial x} \cdot \frac{\partial I}{\partial y} \\ \frac{\partial I}{\partial x} \cdot \frac{\partial I}{\partial y} & \left(\frac{\partial I}{\partial y}\right)^2 \end{bmatrix}$$

$$R = \det M - k(\mathrm{trace}\, M)^2$$

$$\det M = \lambda_1 \lambda_2$$

$$\mathrm{trace}\, M = \lambda_1 + \lambda_2$$

Lucas-Kanade for Optical Flow

- Motion of features across frames due to the relative motion between the scene and the camera.
- Computing Optical Flow of an image can help in Motion Tracking

Lucas-Kanade for Optical Flow

- Consider an Image I(x,y). For smaller motion, the new image can be represented as:

$$H(x,y) = I(x+u,y+v)$$

- Where (u,v) represents the displacement of the pixel.

- Solving this equation using Taylors Expansion we obtain the Lucas-Kanade equation :

$$\begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}$$

$A^T A$ $\qquad$ $A^T b$

Algorithm

Step 1 : Compute the Image x and Image y derivatives.

Step 2 : Compute the difference Image It = Image 1 – Image 2.

Step 3 : Smoothen the image components Ix, Iy and It.

Step 4 : Solve the Linear Equations for each pixel and calculate the Eigen values.

Step 5 : Depending on Eigen values obtained, solve the equations using Cramer's rule.

Step 6 : Plot the optical Flow vectors.